pe_row — Verification & Engineering Log

Project: WS-INT8 Systolic Array Accelerator **Scope of this document:** `pe_row.v` (row of `NUM_PE=3` PEs), current status as of this log's last update **Status:** LIVING DOCUMENT — T1–T5 implemented and passing per `pe_row_spec.md` §6; 27/27 regression directly witnessed via terminal screenshot (Fig. 7, §1). Update as work progresses; do not backdate or embellish entries. **Author:** Jayrajsinh Chauhan | **Mentors:** Prakash Darji, Vandana Choksi (Cadence)

How to use this document: Every claim below is labeled with the evidence that actually supports it — a screenshot I can see, a value I independently re-derived by hand, or something reported to me but not directly witnessed in this session. When reusing this content in a README, CV, SOP, or paper draft, preserve that distinction.

1. Environment & Toolchain Disclosure

Environment	Verilator version	Notes
Local WSL2/Ubuntu (Jayrajsinh's machine)	5.050 (2026-07-01 rev, v5.050-60-g3d2421f3b)	Primary development environment. Source of truth for `rtl/pe_row.v`, `rtl/processing_element.v`, `tb/tb_pe_row.v`.

Unlike the PE-level log, there is **no sandbox-vs-local discrepancy to track here** — all `pe_row` evidence in this document originates from the local WSL environment only. This is a genuine simplification over the PE-level history, not an oversight: no AI-sandbox regression of `pe_row` has been run in parallel, so there is nothing to reconcile.

Evidence actually reviewed this session:

1 terminal screenshot: `verilator --lint-only -Wall rtl/pe_row.v rtl/processing_element.v tb/tb_pe_row.v` (Fig. 1)
5 GTKWave screenshots, one per §6 verification category (Fig. 2–6, mapped to T1–T5 in §4)
1 terminal screenshot: `vvp sim_pe_row` regression run, showing every per-test `PASS` line and the closing summary (Fig. 7)
The current `tb_pe_row.v` source (reviewed directly, not via screenshot)
The current `pe_row.v` source (reviewed directly, not via screenshot)

Terminal summary — directly witnessed. Fig. 7 shows `TOTAL: 27 | PASSED: 27 | FAILED: 0` and `REGRESSION: PASS` from a local WSL run of `vvp sim_pe_row`. This is a genuine witnessed result, not only a source-code cross-check.

One detail Fig. 7 surfaces on its own: it prints **28** `PASS` lines (Basic x1–x3, Reload x1–x5, MAXPOS_C1–C4, MAXNEG_C1–C4, Reset x1–x2, `Reset state`, Reset x3–x4, Timing LOAD, Timing C1–C6), yet the counter reports `TOTAL: 27`. This is not an inconsistency in the tool — it independently confirms the gap already identified in §7: the `Reset state` check is printed via its own bare `$display`, formatted to look identical

to a `report_case` line, but it never calls `report_case` or increments `tests`/`passed`. 28 printed lines minus that 1 uncounted line reconciles exactly to 27. This is a case of a screenshot corroborating a source-read finding rather than merely restating a claimed number.

2. RTL Under Test — Summary

File: `pe_row.v` — reviewed directly this session.

Parameters: `DATA_WIDTH=8`, `ACC_WIDTH=20`, `NUM_PE=3` (default), generatively parameterized — no hardcoded PE count inside the `generate` block or the output assigns.
Built by `generate`-loop instantiation of `processing_element` (reuse constraint, `pe_row_spec.md` §4.1) — confirmed by direct read: no multiply or accumulate logic appears anywhere in `pe_row.v` itself.
`psum_chain` is a `[0:NUM_PE]` wire array: `psum_chain[0] = '0` (identity element, §4.3), `psum_chain[i+1]` = PE `i`'s registered `psum_out`, row output = `psum_chain[NUM_PE]`.
`activation_in` is wired identically (`.activation_in(activation_in)`) to every PE instance — broadcast, not chained (§4.2). Each PE's own `activation_out` is sunk into an otherwise-unused `pe_activation_out` array, explicitly lint-waived (`UNUSEDSIGNAL`) rather than left to generate an unexplained warning.
`overflow_flag = |pe_overflow` — bitwise OR-reduction across all `NUM_PE` individual PE overflow flags (§4.5).
`weight_in_flat` bit-slicing convention: PE `i` uses `weight_in_flat[DATA_WIDTH*(i+1)-1 : DATA_WIDTH*i]` — i.e., PE 0 occupies the **least-significant** slice, PE `(NUM_PE-1)` the most-significant. Combined with `load_row_weights`'s Verilog concatenation `{w_last, ..., w1, w0}` (MSB-first), this means a call like `load_row_weights({8'sd4, 8'sd3, 8'sd2})` places **2→PE0, 3→PE1, 4→PE2** — confirmed by direct cross-read of both files, not inferred.

3. Design Decisions Log (as actually implemented, `pe_row_spec.md` §8)

Decision	Implementation	Rationale (as documented/derivable)
Activation distribution (§4.2)	Single `activation_in` port wired identically to every PE instance in the `generate` loop	Broadcast, not chained — chaining `activation_out→activation_in` would compute a different function (each PE seeing a delayed activation) than the sliding-window sum §4.4 defines. Confirmed structurally: no PE's `activation_out` feeds another PE's `activation_in` anywhere in the file.
First PE's incoming psum (§4.3)	`psum_chain[0] = '0`	0 is the additive identity — PE 0 has no upstream partial sum, so it must start its accumulation from a value that leaves its own product unchanged.

Decision	Implementation	Rationale (as documented/derivable)
`weight_in_flat` bit-slice convention (§5)	PE `i` ← `weight_in_flat[DATA_WIDTH*(i+1)-1 : DATA_WIDTH*i]` (PE 0 = LSB slice)	Documented in the header comment and matched exactly in `tb_pe_row.v`'s `load_row_weights` packing — the exact self-inflicted-bug the spec warns about (DUT/TB convention mismatch) was avoided by using one packed input and one slicing rule, rather than separate positional TB arguments that could silently diverge from the RTL's indexing.
Overflow combining logic (§4.5)	`overflow_flag = |pe_overflow` (reduction-OR over a `[NUM_PE-1:0]` vector)	Reduction-OR is correct for "any of N single-bit conditions is true" for *any* N — it doesn't need to be re-derived if `NUM_PE` changes, satisfying the spec's explicit generality requirement.

4. Current Test Suite — Status (Local WSL, `tb_pe_row.v` current source)

File under test: `tb_pe_row.v`, reviewed directly this session. Uses the packed-array `load_row_weights(input signed [NUM_PE*DATA_WIDTH-1:0] weights)` task — **no hardcoded 3-argument version remains.** This closes the structural debt item flagged when `pe_row` testbench work began (positional `w0,w1,w2` args would have blocked reuse at other `NUM_PE` values); the packed-bus form is generic by construction.

Test	`pe_row_spec.md` §6 category	Result claimed	Screenshot reviewed	My independent hand-check
T1	1. Basic correctness	PASS	Fig. 2 (WF-1)	✅ re-derived, see §5.1
T2	2. Weight reload mid-sequence	PASS	Fig. 3 (WF-2)	✅ re-derived, see §5.2
T3	3. Overflow (redesigned as non-overflow confirmation — see note)	PASS	Fig. 4 (WF-3, two boundary loads)	✅ re-derived, see §5.3
T4	4. Reset mid-stream	PASS	Fig. 5 (WF-4)	✅ structurally consistent with PE-level reset (§5.4)
T5	5. Timing check vs. §4.4 table	PASS	Fig. 6 (WF-5)	✅ re-derived, see §5.5

Note on T3: As already documented in the timing-table work, with `NUM_PE=3` and `ACC_WIDTH=20`, the maximum achievable row-level `|psum|` from legal INT8 inputs is `3 × 16384 = 49152` — far below the signed 20-bit boundary (`±524288`). Overflow is architecturally unreachable at this `NUM_PE` until `NUM_PE ≥ 32`. T3 was correctly redesigned to *confirm* this (drive both sign-direction magnitude extremes, assert

`overflow_flag` stays 0), rather than force an overflow that the current configuration cannot produce. This is a documented scope decision, not a weakened test.

Lint: Fig. 1 shows `verilator --lint-only -Wall` on `rtl/pe_row.v rtl/processing_element.v tb/tb_pe_row.v`, Verilator 5.050, local WSL. The visible output is the standard 3-line summary banner (Verilation Report / Verilator built from N sources / walltime) with **no warning lines above it** — consistent with a clean run. As with the PE-level log's §1 caution: this is a clean *visible* result; if you want it citable with the same rigor as "0 warnings, confirmed," keep the full untruncated terminal scrollback saved, since a screenshot can in principle be cropped above earlier warning lines. I have no reason to think that happened here, but I'm naming the same caveat you insisted on for the PE-level `BLKSEQ`/`SYNCASYNCNET` discrepancy so the standard is applied consistently.

5. Independent Hand Re-Derivation (this session)

I re-derived the row's cycle-by-cycle behavior directly from `pe_row.v`'s actual wiring (registered chain, one-cycle latency per stage — same governing rule as `pe_row_timing_table.md`), independent of the testbench's stated expected values, and checked it against every test category. This is a genuine cross-check, not a restatement of your numbers.

Model used: for PE `i` at cycle `c`, with `w[i]` stationary and broadcast activation `x[c]`: `PEi_out(c) = PEi-1_out(c-1) + w[i]·x[c]`, with `PE(-1)_out ≡ 0` for all cycles (PE0's fixed upstream), and register state persisting across `load_row_weights` cycles (loading doesn't clear the psum registers — only reset does).

5.1 T1 — Basic correctness

Weights `{4,3,2}` → PE0=2, PE1=3, PE2=4 (per §2 slicing). Starting from reset (all psum regs 0):

`x=1`: PE0=2, PE1=0+3=3, PE2=0+4=4 → row=**4** ✅ matches
`x=2`: PE0=4, PE1=2+6=8, PE2=3+8=11 → row=**11** ✅ matches
`x=-1`: PE0=-2, PE1=4-3=1, PE2=8-4=4 → row=**4** ✅ matches

5.2 T2 — Weight reload mid-sequence

Continuing from T1's final register state (PE0=-2, PE1=1, PE2=4 — **not** reset), same weights reloaded, then a new set `{7,6,5}` → PE0=5,PE1=6,PE2=7:

`x=1` (weights still 2,3,4): PE0=2, PE1=-2+3=1, PE2=1+4=**5** ✅ matches
`x=2`: PE0=4, PE1=2+6=8, PE2=1+8=**9** ✅ matches
reload `{7,6,5}`, `x=1`: PE0=5, PE1=4+6=10, PE2=8+7=**15** ✅ matches
`x=2`: PE0=10, PE1=5+12=17, PE2=10+14=**24** ✅ matches
`x=-1`: PE0=-5, PE1=10-6=4, PE2=17-7=**10** ✅ matches

This confirms two things beyond "the numbers matched": (1) reload correctly changes subsequent products without corrupting the in-flight chained partial sums already sitting in PE1/PE2's registers, and (2) the row genuinely does **not** re-zero on a weight-only load — only `rst_n` does (T4 territory).

5.3 T3 — Boundary / non-overflow confirmation

Weights `{-128,-128,-128}` (all PEs), `x=-128` held for 4 cycles → each PE's own product = `16384` every cycle:

c1: PE0=16384, PE1=0+16384=16384, PE2=0+16384=**16384** ✅
c2: PE0=16384, PE1=16384+16384=32768, PE2=16384+16384=**32768** ✅
c3: PE0=16384, PE1=32768, PE2=32768+16384=**49152** ✅
c4 (steady state): PE2 stays **49152** ✅ — matches the architectural ceiling `3×16384=49152` cited above; `overflow_flag=0` throughout is correct and expected, not a false negative.

The mirrored negative-product case (`{127,127,127}` weights, `x=-128` → each product `-16256`) follows the same structure to `-48768`, symmetric to the positive case. I did not re-derive every intermediate digit of the negative case by hand in this pass, but the structure is identical to the positive case above with sign flipped, and the reported values (`-16256, -32512, -48768, -48768`) are internally consistent with that structure (each step adds exactly `-16256`, and step 3→4 repeats, as expected once the row has filled).

5.4 T4 — Reset mid-stream

Structurally, `pe_row`'s reset behavior is not independent logic — it is inherited entirely from each instantiated `processing_element`'s own async active-low reset (already verified at the PE level, §1 of the PE log: `weight_reg`, `activation_out`, `psum_out`, `overflow_flag` all clear to 0). Since `pe_row` adds no registers of its own (only combinational wiring/`generate` structure), asserting `rst_n=0` mid-stream on the row necessarily drives every constituent PE's registers to 0 simultaneously, which is exactly the documented row-level "known state" (`psum_out=0, overflow_flag=0`). This is a case where PE-level verification *does* transfer to the row for reset specifically, because reset touches no row-level composition logic — unlike functional correctness (T1/T2/T5), which genuinely exercises new row-level wiring and had to be independently checked.

5.5 T5 — Timing check vs. §4.4 table

Weights `{4,3,2}` (PE0=2,PE1=3,PE2=4), fresh reset, stream `x = 1,2,-1,0,1,2`:

LOAD phase: `psum_out=0, overflow_flag=0` ✅ (registers untouched during `load_compute_sel=0`)
C1 (`x=1`): row=**4** ✅
C2 (`x=2`): row=**11** ✅
C3 (`x=-1`): row=**4** ✅
C4 (`x=0`): PE0=0, PE1=-2+0=-2, PE2=1+0=**1** ✅
C5 (`x=1`): PE0=2, PE1=0+3=3, PE2=-2+4=**2** ✅
C6 (`x=2`): PE0=4, PE1=2+6=8, PE2=3+8=**11** ✅

All 6 compute-cycle values plus the LOAD-phase value re-derive exactly to what the testbench expects, using only the registered-chain model — this is the strongest single piece of evidence in this log, since T5 is explicitly the test the spec calls out as catching RTL/testbench-boundary bugs that a single final-value check would miss.

6. Verification History Note (carried forward from `pe_row_timing_table.md`)

The §4.4 timing table itself required **two correction rounds** before being accepted (documented in `pe_row_timing_table.md`, "Verification history"): first for missing the carried upstream term entirely, then for activation-index errors and a stale-value repeat at specific cells. This is worth keeping in any CV/SOP narrative — it is honest evidence of the predict-then-verify discipline actually catching design-table errors *before* they became RTL bugs, not after.

7. Quantitative Evidence Summary

27 total self-checked test cases in `tb_pe_row.v`, distributed as: T1=3, T2=5, T3=8 (4 positive-boundary + 4 negative-boundary), T4=4 (excluding one un-counted manual `$display`-only reset check — see note below), T5=7 (1 explicit LOAD-phase check + 6 compute-cycle checks). This count was obtained by manually enumerating `report_case`/explicit `tests=tests+1` call sites in the reviewed source, **cross-checked against**, not substituted for, your reported terminal figure.
1 uncounted check exists: Test 4 contains a `psum_out`/`overflow_flag` reset-state check that uses a bare `$display("PASS"/"FAIL" ...)` without calling `report_case` or incrementing `tests`. It is real verification (traced structurally in §5.4) but does **not** contribute to the `tests`/`passed`/`failed` counters. **Confirmed directly, not just inferred from source:** Fig. 7's terminal output prints 28 `PASS` lines against a reported `TOTAL: 27` — the `Reset state` line is exactly the one that doesn't reach the counter. Worth knowing if the 27-count is ever scrutinized line-by-line, since a careful reviewer could ask "why isn't the mid-stream reset assertion itself counted?"
6 of 6 T5 compute-cycle values, all 5 T2 values, and all 3 T1 values independently re-derived by hand this session using the registered-chain model, all matching (§5).
0 RTL changes required to `pe_row.v` during this verification pass, per your report (not independently confirmed via a diff or checksum in this session, unlike the PE-level log's md5sum discipline — worth adopting the same practice here if you want equal rigor).
Verilator 5.050, local WSL, `--lint-only -Wall`: clean on the combined RTL+TB — confirmed via screenshot (Fig. 1).

8. Engineering Terminology / Skills — Classification

Activity / Term	Classification	Basis
RTL composition by instantiation (`generate`-loop hierarchy)	✅ ACTUALLY PERFORMED	`pe_row.v` reviewed directly, §2
Parameterized/generative design (no hardcoded `NUM_PE`)	✅ ACTUALLY PERFORMED	Confirmed via direct read — `generate for` loop keyed off `NUM_PE`, output assigns index `psum_chain[NUM_PE]`

Activity / Term	Classification	Basis
Interface/bus packing and slicing convention design	✅ ACTUALLY PERFORMED	`weight_in_flat` slicing documented and matched DUT-to-TB, §2–3
Design-decision documentation (spec §8 decisions log)	✅ ACTUALLY PERFORMED	§3, sourced from actual header-comment intent and RTL structure
Cycle-accurate timing analysis / pipeline latency reasoning	✅ ACTUALLY PERFORMED	`pe_row_timing_table.md`, independently re-derived this session, §5
Directed testbench development against a written verification plan (§6 categories)	✅ ACTUALLY PERFORMED	`tb_pe_row.v`, 5 categories implemented, §4
Predict-then-verify discipline	✅ ACTUALLY PERFORMED	Hand-derivation shown to independently reproduce every T1/T2/T5 value, §5
Architectural scoping analysis (overflow-unreachability at given `NUM_PE`)	✅ ACTUALLY PERFORMED	T3 redesign rationale, §4 note
Reset-behavior inheritance reasoning (row vs. PE-level)	✅ ACTUALLY PERFORMED	§5.4 — correctly identified as a case where PE-level proof *does* transfer, unlike functional composition
Static lint on a hierarchical design (row + instantiated PE + TB)	✅ ACTUALLY PERFORMED	Fig. 1, local WSL, Verilator 5.050
Waveform-based confirmation of a timing table	✅ ACTUALLY PERFORMED	Fig. 2–6 reviewed; T5 specifically maps to the §4.4 table, §4–5
Source-controlled checksum/diff discipline on RTL during a verification pass	⬜ NOT YET PERFORMED for `pe_row.v`	Adopted at PE level (md5sum before/after G1–G16); not yet replicated here — see §9
Functional/code coverage collection	⬜ NOT YET PERFORMED	Same open item as PE-level log (G17-equivalent)
Regression scripting (Makefile-driven lint→sim→report for `pe_row`)	🟡 PARTIALLY PERFORMED	Project Makefile exists per prior sessions; not confirmed exercised specifically for `pe_row` within this document's evidence base
Synthesis / STA / PPA analysis	⬜ NOT YET PERFORMED	Planned for post-row, 2D-array DSE phase per project scope

9. Next Steps (as of this log entry)

~~Close the evidence gap named in §1.~~ **Done** — Fig. 7 (`vvp sim_pe_row`, local WSL) shows the full `TOTAL: 27 | PASSED: 27 | FAILED: 0` / `REGRESSION: PASS` output, directly witnessed.
Adopt the PE-level log's md5sum-before/after discipline for `pe_row.v` during future testbench-only changes, so "0 RTL changes during this verification pass" becomes independently checkable rather than reported.
Decide whether the uncounted Test 4 reset-state `$display` check (§7, now confirmed via Fig. 7's 28-vs-27 line count) should be folded into `report_case` / `tests` for consistency, or explicitly documented as an intentionally-separate sanity check — right now it's an unlabeled inconsistency a reviewer could flag.
Move to `systolic_array.v` (2D array) design per the project roadmap — `pe_row` is composition-verified (T1/T2/T5 independently re-derived, T3/T4 structurally sound, 27/27 regression witnessed) and ready to be treated as the next reusable building block, the same way `processing_element` was treated once its own verification closed.
Before the symposium submission, revisit the still-open PE-level items (G17 coverage, G18 recorded waveform review) and consider whether `pe_row` needs its own coverage/waveform-review pass at the same standard, rather than only inheriting the PE-level gap list by association.

Last updated: this session, from user-provided lint screenshot, five GTKWave screenshots, a terminal regression-summary screenshot (Fig. 7), and direct review of `pe_row.v` / `tb_pe_row.v` source. Do not silently overwrite prior evidence without a record of what changed.